

for reading and manipulation.

```
var sess;
app.get('/', function(req, res) {
  sess = req.session;
  /*
```

The session is assigned to the variable `sess` here. And it can now be manipulated by creating and assigning new session variables.

```
*/
//sample session variables
sess.email;
sess.username;
});
```



Note: In this sample code, we are creating a session variable 'sess', and using the email and the user name as two attributes. In the final code at the end, we will use different variables (email and visits) to show the working of the session.

Project structure

The server code will be in `index.js`, and `home.html` will be the main landing page where the login form will be displayed.

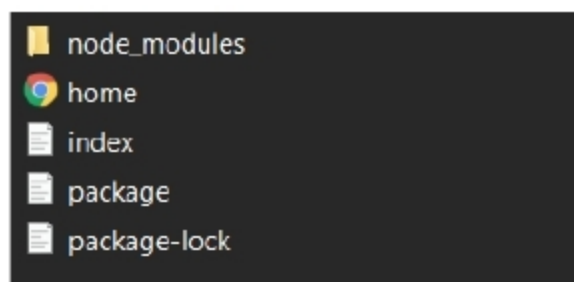


Figure 3: Node.js file structure

Here is the server-side code in `index.js`:

```
//index.js
//including the imported packages
const express = require('express');
const session = require('express-session');
const bodyParser = require('body-parser');

//initialize the app as an express app
const app = express();
```

```
const router = express.Router();

//middleware
app.use(
  session({
    secret: 'thisisasecret',
    saveUninitialized: false,
    resave: false
  })
);
app.use(bodyParser.json());
app.use(
  bodyParser.urlencoded({
    extended: true
  })
);

var sess; //is a global variable, NOT RECOMMENDED!

//ROUTERS

//Router 1: for rednering the hompage
router.get('/', (req, res) => {
  sess = req.session;
  if (sess.email && sess.visits) {
    return res.redirect('/admin');
  } else {
    sess.visits = 1;
    res.sendFile(__dirname + '/home.html');
  }
});
```

```
//Router 2: for login operation
router.post('/login', (req, res) => {
  sess = req.session;
  sess.email = req.body.email;
  res.end('done');
});
```

```
// Router 2: goes to a page that can only be accessed
// if the user is logged in.
router.get('/admin', (req, res) => {
  sess = req.session;
  if (sess.email) {
    res.write(`<h1>Hello ${sess.email}</h1><br>`);
    res.write(
      `<h3>This is visit number ${sess.visits}</h3><h5>Refresh page to increase`
```

```
visits</h5>`
    );
    sess.visits++;
    res.end(`<a href= ' + '/logout' + '>Click here to log out</a>`);
  } else {
    res.write(`<h1>You need to log in before you can see this page.</h1>`);
    res.end(`<a href= ' + '/' + '>Login</a>`);
  }
});

//Router 4: for session destruction
router.get('/logout', (req, res) => {
  req.session.destroy(err => {
    if (err) {
      return console.log(err);
    }
    res.redirect('/');
  });
});

app.use('/', router);
app.listen(process.env.PORT || 3000, () => {
  console.log(`App Started on PORT ${process.env.PORT || 3000}`);
});
```

Each router first checks if the variable user name is set in the session that can only be set by logging in through the front-end.

This will be our HTML template for the front-end, and will be used for logging the client in and out. It is a very plain and basic HTML template without any CSS, and this is only being used for demo purposes.

```
<html>
<head>
  <title>Testing Session</title>
</head>
<body>
  Username:
  <input type="text" id="email" /><br />
  Password :
  <input type="password" id="password" /><br />
```